so before proceeding any further. You can find compatibility and support information at *https://www.microsoft.com/net/download/dotnet-core/2.1*.

Let's create a directory called 'mnist' anywhere you like, and execute the following commands to create a .NET Core console app inside that directory and to add the ML.NET framework to our project:

```
dotnet new console
dotnet add package Microsoft.ML
```

We can now move ahead on our journey to machine learning in .NET. Fire up your favourite text edit and open the 'Program.cs' file in your project directory. You will see the following 'Hello World!' code shown below. Let's start with this and change it as we move forward:

```
using System;

namespace mnist
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

If you are trying to build a .NET Core app for the first time, you can also build and run it by running 'dotnet run' in the project directory. Moving forward, the first order of business is to add all the namespaces that we will need to get this project running. Add the following lines to the top of the *Program.cs* file, as follows:

```
using System;
using System.IO;
using System.Linq;
using Microsoft.ML;
using Microsoft.ML.Core.Data;
using Microsoft.ML.Runtime.Api;
using Microsoft.ML.Runtime.Data;
using Microsoft.ML.StaticPipe;
using System.Threading.Tasks;
```

The next thing we need to set up is a few variables to point to our training and test data set, as well as the name and path of our ML model to save once we are done. Add the following lines under the class *Program* just before the *Main()* method:

```
        static readonly string _datapath = Path.
Combine(Environment.CurrentDirectory, "Data", "optdigits-
train.csv");
        static readonly string _testdatapath = Path.
Combine(Environment.CurrentDirectory, "Data", "optdigits-
test.csv");
        static readonly string _modelpath = Path.
Combine(Environment.CurrentDirectory, "Data", "Model.zip");
```

Data processing

You may have noticed that we have referred to a directory called *Data* in the code snippet above. This directory will host our MNIST data files as well as the trained model once we are done. Please download the MNIST 8x8 data set* from the UCI Machine Learning Repository at *http://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits*.

Click on the 'Data Folder' link on the page and download the following files in a directory called *Data* in your project directory:

```
optdigits.tra
optdigits.tes
```

Just to make it easier for us, we will rename these files to *optdigits-train.csv* and *optdigits-test.csv*. If you haven't yet looked at them, this is a good time to do so.

The MNIST data set we are using contains 65 columns of numbers. The first 64 columns in each row are integer values in the range from 0 to 16. These values are calculated by dividing 32 x 32 bitmaps into non-overlapping blocks of 4 x 4. The number of ON pixels is counted in each of these blocks, which generates an input matrix of 8 x 8. The last column in each row is the number that is represented by the values in the first 64 columns. These columns are our features and the last column is our label, which we will predict using our ML model once we are finished with this project.

It's time to set up the data classes that we will use when training and testing our ML model. To do that, let's add the following classes under the *mnist* namespace in our *Program.cs* file, as follows:

```
 public class MNISTData
{
    [Column("0-63")]
    [VectorType(64)]
    public float[] PixelValues;

    [Column("64")]
    public string Number;
}

public class NumberPrediction
{
    [ColumnName("PredictedLabel")]
    public string NumberLabel;
```